Comparar la implementación de un diccionario de datos físico en diferentes SGBD

Por:

Johan Darío Martínez Echeverri

Jonathan Montoya Machado

Tecnología en Desarrollo de Software

G100 (SD1006) Bases de Datos I

Oralia Cortes Grajales

Institución Universitaria Pascual Bravo

Medellín, Colombia

2026-2

1. **Justificación de la Elección de los Tres SGBD**

Para el desarrollo de la base de datos de la **Red Social Estudiantil Pascualina**, se seleccionaron tres motores relacionales de alto impacto en la industria software, cada uno representativo de un paradigma arquitectónico distinto:

- **PostgreSQL 17:** Representa el estándar de los SGBD objeto-relacionales de código abierto. Se eligió como el motor principal debido a su arquitectura extensible, la potencia del tipo de dato binario JSONB para cargas semiestructuradas y el soporte nativo del ecosistema **PostGIS**, considerado el estándar global para el procesamiento de información geográfica y espacial.
- **MySQL 8.0:** Es el SGBD relacional de código abierto más desplegado en entornos web a nivel mundial. Su selección permite evaluar la portabilidad del modelo a infraestructuras web livianas, comprobando el comportamiento del tipo de dato JSON nativo y de los índices espaciales (SPATIAL INDEX sobre tipos POINT) bajo el motor de almacenamiento InnoDB.
- **Microsoft SQL Server 2022:** Corresponde al entorno propietario empresarial del ecosistema Microsoft. Su inclusión justifica la evaluación de reglas de integridad referencial estrictas (como el manejo de ciclos o rutas múltiples de borrado en cascada) y el almacenamiento de JSON mediante el tipo NVARCHAR(MAX) validado por restricciones de verificación (ISJSON()).

2. **Explicación paso a paso de la implementación del diccionario de datos físico en cada SGBD.**

Al traducir el modelo físico entre los tres SGBD, los atributos de cada tabla sufrieron transformaciones directas debido a la forma en que cada motor gestiona el almacenamiento, los autoincrementales, las fechas, los datos semiestructurados (NoSQL) y la geolocalización (IoT).

- **Identificadores Autoincrementales y Secuencias**

  PostgreSQL: Utiliza el pseudo-tipo SERIAL, el cual crea implícitamente una secuencia (sequence) adjunta al campo y define el valor por defecto como nextval('tabla_id_seq'::regclass).

  MySQL: Emplea el atributo de columna AUTO_INCREMENT sobre claves primarias enteras. Requiere el motor InnoDB para manejar la concurrencia de bloqueos en la tabla.

  SQL Server: Utiliza la propiedad de columna IDENTITY(1,1), donde se define explícitamente el valor inicial (semilla) y el incremento. A diferencia de MySQL y PostgreSQL, no permite insertar valores manuales en dicho campo a menos que se active SET IDENTITY_INSERT ON.

- **Fechas y Marcas de Tiempo**

  PostgreSQL: El tipo TIMESTAMP (o TIMESTAMP WITHOUT TIME ZONE) almacena fecha y hora con precisión de microsegundos (6 dígitos decimales). Se asigna DEFAULT CURRENT_TIMESTAMP.

  MySQL: Se utiliza DATETIME, el cual soporta el rango de '1000-01-01' a '9999-12-31'. El valor por defecto se configura mediante DEFAULT CURRENT_TIMESTAMP.

  SQL Server: Se reemplazó el tipo antiguo DATETIME (precisión de 3.33 ms) por DATETIME2, ofreciendo una precisión de hasta 100 nanosegundos (7 dígitos decimales) y un menor consumo de almacenamiento, utilizando DEFAULT GETDATE().

- **Campos Semiestructurados (NoSQL / JSON)**

  PostgreSQL: Emplea JSONB (JSON Binario). Los datos se convierten a un formato binario descompuesto al insertar, lo que elimina espacios y llaves duplicadas, permitiendo indexación avanzada mediante GIN. Se define el valor por defecto como '{}'::jsonb.

  MySQL: Utiliza el tipo nativo JSON. Valida que el documento ingresado sea sintácticamente correcto y lo almacena de forma optimizada, aunque no utiliza el formato binario descompuesto de PostgreSQL.

  SQL Server: Al carecer de un tipo de dato dedicado denominado JSON, se implementa mediante NVARCHAR(MAX) combinado con una restricción CHECK utilizando la función ISJSON(columna) > 0. Esto garantiza la integridad del objeto NoSQL sin perder compatibilidad.

- **Tipos de Datos Espaciales e IoT**
  incorporamos un módulo de datos espaciales en eventos y mentorías. Capturan las coordenadas GPS de los eventos y las ubicaciones de check-in de los estudiantes. El campo ubicacion permite realizar consultas de proximidad espacial para determinar la distancia exacta entre los estudiantes y las aulas o bloques donde se desarrollan las actividades.

  PostgreSQL: Requiere activar la extensión PostGIS (CREATE EXTENSION IF NOT EXISTS postgis). Utiliza el tipo GEOGRAPHY(POINT, 4326), el cual calcula distancias esféricas sobre el elipsoide terrestre WGS84.

  MySQL: Utiliza el tipo espacial nativo POINT SRID 4326. A diferencia de PostgreSQL, la definición del SRID se incluye directamente en la especificación del campo de la columna en la instrucción CREATE TABLE.

SQL Server: Implementa el tipo de dato espacial de sistema GEOGRAPHY. Mantiene internamente el sistema de referencia espacial (por defecto SRID 4326 si no se especifica otro) y permite ejecutar métodos OGC directamente sobre la columna (.STDistance(), .STAsText()).

- **Integridad Referencial y Manejo de Cascadas**

  PostgreSQL y MySQL: Soportan reglas complejas de ON DELETE CASCADE en relaciones autorreferenciales o múltiples (como las tablas seguir y mentoria).

  SQL Server: Su motor relacional aplica una restricción estricta contra múltiples rutas de cascada o ciclos (Error 1785). Para resolver esto, en SQL Server se debieron modificar las claves foráneas de seguir y mentoria de ON DELETE CASCADE a ON DELETE NO ACTION.

- **Extracción Técnica del Diccionario de Datos Físico Paso a Paso:**

  PostgreSQL:
  los metadatos se consultan combinando las vistas del information_schema. El tipo SERIAL se refleja en la columna column_default como una función de secuencia

  Ver Diccionario de datos:

```
SELECT
column_name AS "Columna",
data_type AS "Tipo de Dato Fisico",
character_maximum_length AS "Longitud Max",
is_nullable AS "Permite Nulos",
column_default AS "Valor por Defecto"
FROM
information_schema.columns
WHERE
table_schema = 'public'
AND table_name = 'NOM_TABLA' -- Reemplaza con el nombre de tu tabla
ORDER BY ordinal_position;
```

  Resultado de la extracción en PostgreSQL:

  data_type: Retorna integer, character varying, timestamp without time zone, USER-DEFINED (para el tipo geography de PostGIS) o jsonb.

column_default: Muestra explícitamente expresiones como '{}'::jsonb o nextval('evento_id_evento_seq'::regclass).

MySQL 8.0:

se consulta INFORMATION_SCHEMA.COLUMNS especificando el esquema mediante la función DATABASE(). Se debe solicitar el campo COLUMN_TYPE para ver la longitud real de tipos numéricos y cadenas, y EXTRA para detectar atributos autoincrementales.

Comando para ver el diccionario de datos:

```
SELECT
   COLUMN_NAME AS 'Columna',
   COLUMN_TYPE AS 'Tipo de dato físico',
   CHARACTER_MAXIMUM_LENGTH AS 'Longitud máxima',
   IS_NULLABLE AS 'Permite nulos',
   COLUMN_DEFAULT AS 'Valor por defecto',
   COLUMN_KEY AS 'Clave',
   EXTRA AS 'Atributos adicionales'
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = DATABASE()
  AND TABLE_NAME = = 'NOM_TABLA' -- Reemplaza con el nombre de tu tabla
ORDER BY ORDINAL_POSITION;
```

Resultado de la extracción en MySQL:

COLUMN_TYPE: Devuelve la definición completa utilizada por InnoDB, por ejemplo int, varchar(100), point, json.

EXTRA: Retorna la cadena auto_increment cuando aplica.

COLUMN_KEY: Muestra PRI (Primary Key), UNI (Unique) o MUL (Key indexada/Foreign Key)[cite: 5].

Microsoft SQL Server

En SQL Server, INFORMATION_SCHEMA.COLUMNS extrae la información estándar, pero para detectar las propiedades de identidad (IDENTITY) y los

tipos espaciales exactos se complementa con las vistas del sistema sys.columns y sys.tables.

Ver Diccionario de datos:

SELECT

CAST(TABLE_NAME AS VARCHAR(25)) AS Tabla,

CAST(COLUMN_NAME AS VARCHAR(25)) AS Columna,

CAST(DATA_TYPE AS VARCHAR(18)) AS Tipo_Dato,

CHARACTER_MAXIMUM_LENGTH AS Longitud,

CAST(IS_NULLABLE AS VARCHAR(5)) AS Nulos

FROM INFORMATION_SCHEMA.COLUMNS

WHERE TABLE_NAME = ' NOM_TABLA ' -- Reemplaza con el nombre de tu tabla

ORDER BY ORDINAL_POSITION;

Resultado de la extracción en SQL Server:

Tipo de Dato Físico: Devuelve int, varchar, datetime2, geography, nvarchar.

Atributos Extra: Identifica los campos controlados por el motor mediante IDENTITY(1,1).

- **Comparativa del Diccionario de Datos Físico (tabla evento)**

Los diccionarios de datos físico contempla la totalidad de las entidades que componen la Red Social Estudiantil Pascualina en los tres SGBD seleccionados: PostgreSQL, MySQL y Microsoft SQL Server.

En este documento se presenta en detalle la tabla evento como caso de estudio representativo, por integrar la combinación completa de tipos de datos lógicos, claves primarias autoincreméntales, llaves foráneas con borrado en cascada, metadatos NoSQL dinámicos (datos_extra) y coordenadas GPS espaciales (ubicacion).

Campo (Lógico)	**Métrica / Clave**	**PostgreSQL 17**	**MySQL 8.0**	**SQL Server 2022**	**Descripción Física**
id_evento	**PK**	SERIAL (integer)	INT AUTO_INCREMENT	INT IDENTITY(1, 1)	Clave primaria autoincremental

Campo (Lógico)	Métrica / Clave	PostgreSQL 17	MySQL 8.0	SQL Server 2022	Descripción Física
					única del evento.
id_grupo	**FK**	INT (ON DELETE CASCADE)	INT (ON DELETE CASCADE)	INT (ON DELETE CASCADE)	Relación con la tabla grupo.
nombre	Requerido	VARCHAR(100)	VARCHAR(100)	VARCHAR(100)	Nombre oficial del evento estudiantil.
descripcion	Opcional	VARCHAR(500)	VARCHAR(500)	VARCHAR(500)	Detalle o agenda del evento.
fecha_inicio	Requerido	TIMESTAMP	DATETIME	DATETIME2	Fecha y hora programada de inicio.
fecha_fin	Requerido	TIMESTAMP	DATETIME	DATETIME2	Fecha y hora programada de finalización.
lugar	Opcional	VARCHAR(150)	VARCHAR(150)	VARCHAR(150)	Descripción textual del aula o bloque.

Campo (Lógico)	Métrica / Clave	PostgreSQL 17	MySQL 8.0	SQL Server 2022	Descripción Física
modalidad	Requerido	VARCHAR(30)	VARCHAR(30)	VARCHAR(30)	Presencial, Virtual o Híbrida.
organizador	Requerido	VARCHAR(100)	VARCHAR(100)	VARCHAR(100)	Entidad o líder responsable.
ubicacion	Espacial	GEOGRAPHY(POINT,4326)	POINT SRID 4326	GEOGRAPHY	Coordenadas GPS del evento
datos_extra	NoSQL	JSONB DEFAULT '{}'::jsonb	JSON	NVARCHAR(MAX) + ISJSON()	Metadatos dinámicos (cupos, links, etiquetas).
fecha_creacion	Default	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	DATETIME DEFAULT CURRENT_TIMESTAMP	DATETIME2 DEFAULT GETDATE()	Timestamp automático de auditoría de creación.

3. Dificultades Encontradas, Ventajas y Desventajas Comparativas

PostgreSQL:

Dificultades encontradas durante la implementación:

Durante la implementación de la base de datos de la Red Social Estudiantil Pascualina, una de las principales dificultades fue adaptar los tipos de datos del modelo físico a las características de PostgreSQL. Por ejemplo, fue necesario tener en cuenta el uso de SERIAL para los identificadores autoincrementales y de TIMESTAMP para almacenar fechas y horas.

Otra dificultad fue la configuración de los datos geográficos. Para trabajar con coordenadas y realizar consultas espaciales, se requiere instalar o habilitar la extensión PostGIS en el entorno correspondiente. Además, fue necesario considerar las conversiones de tipos de datos para trabajar correctamente con información de tipo GEOGRAPHY.

También fue necesario comprender el funcionamiento de JSONB, ya que este tipo de dato permite almacenar información semiestructurada, como enlaces, etiquetas y datos adicionales de los eventos, utilizando una estructura flexible sin necesidad de crear nuevas columnas para cada atributo opcional.

Ventajas:

Extensión PostGIS madura para cálculos espaciales.

JSONB almacena JSON binario indexable vía GIN.

Manejo flexible de reglas en cascada autorreferenciales.

Desventajas:

Requiere la activación explícita de la extensión espacial (CREATE EXTENSION IF NOT EXISTS postgis).

Exige *casting* explícito de datos espaciales (::geography)

Algunas características avanzadas, como PostGIS, los operadores de JSONB y la configuración de índices especializados, requieren práctica para utilizarlas correctamente.

SQL Server:

Dificultades encontradas durante la implementación:

Al realizar la traducción DDL hacia Microsoft SQL Server, se identificó la restricción borrados múltiples en cascada (ON DELETE CASCADE) cuando hay rutas circulares (Error 1785), obligando al uso de ON DELETE NO ACTION. Este error ocurre porque la eliminación de un estudiante genera dos rutas simultáneas en cascada sobre las tablas reaccion y comentario: una directa (estudiante - reaccion) y otra indirecta (estudiante - publicacion - reaccion). Para eliminar la ambigüedad y evitar bloqueos en el motor, se reconfiguró la restricción de la llave foránea de id_estudiante a ON DELETE NO ACTION en ambas tablas. Asimismo, se adaptó la sintaxis de indexación espacial empleando USING GEOGRAPHY_AUTO_GRID para el tipo de dato GEOGRAPHY.

Ventajas:
Integración nativa con SQL Server Management Studio (SSMS).

Manejo de tipos espaciales GEOGRAPHY incorporado sin instalar plugins adicionales.

Desventajas:

No posee un tipo de dato nativo llamado JSON; requiere NVARCHAR(MAX) y restricción ISJSON().

Bloquea borrados múltiples en cascada (ON DELETE CASCADE) cuando hay rutas circulares (Error 1785), obligando al uso de ON DELETE NO ACTION.

MySQL:

Dificultades encontradas durante la implementación:

Al llevar el diseño de la red social estudiantil a código, una de las cosas más complicadas fue la restricción en la longitud de los identificadores. Intentar nombrar la base de datos o las tablas con frases muy descriptivas nos arrojó errores inmediatos, ya que el motor tiene un límite estricto de 64 caracteres. Esto obligó a replantear la nomenclatura para que fuera mucho más corta y directa.

Otra de las dificultades fue la gestión de las claves foráneas al ejecutar los scripts en bloque. Si no se respeta el orden jerárquico exacto (creando primero las tablas padres como estudiante o grupo, y dejando para el final las dependencias como miembro_grupo o reaccion), el sistema simplemente bloquea la ejecución. Lo mismo ocurrió con la limpieza del esquema; intentar borrar tablas sin usar correctamente el DROP TABLE IF EXISTS en el orden inverso generaba conflictos de restricciones constantemente.

Ventajas:

Una gran ventaja para este proyecto fue la capacidad de integrar campos de tipo JSON en tablas como perfil, evento y publicación. Esto dio muchísima flexibilidad para

guardar datos dinámicos o semiestructurados sin tener que alterar el esquema relacional creando tablas adicionales cada vez que surgía un nuevo requerimiento.

Por otro lado, la compatibilidad nativa con datos espaciales mediante el tipo POINT facilitó mucho la planificación del módulo de mentorías y eventos físicos. Poder guardar coordenadas geográficas directamente hace que proyectar ubicaciones dentro de la ciudad o el campus sea mucho más directo y eficiente a nivel de consultas.

Desventajas:

A pesar de su popularidad, el manejo de errores en la consola de MySQL puede llegar a ser bastante ambiguo. Cuando hay un fallo de sintaxis (como un espacio mal puesto, un salto de línea invisible o un problema con las comillas), el motor suele arrojar un mensaje genérico que solo indica cerca de qué palabra falló, obligando a revisar todo el bloque de código minuciosamente.

Conclusión:

JDME

El diseño de la base de datos para la Red Social Estudiantil Pascualina en PostgreSQL y SQL Server demuestra que traducir un modelo relacional no es un simple traspaso de sintaxis, sino una adaptación estratégica a las reglas de almacenamiento y rendimiento de cada motor.

Mientras PostgreSQL destaca por su flexibilidad en el manejo de datos semiestructurados con JSONB y el procesamiento de coordenadas GPS con PostGIS, SQL Server impone un control de integridad relacional mucho más riguroso, lo que exigió ajustar las reglas de borrado en cascada para evitar rutas múltiples (Error 1785) y validar la estructura NoSQL mediante la restricción ISJSON().

En conclusión, la integración de datos estructurados, atributos NoSQL y geolocalización para casos de uso IoT permitió construir una base de datos multimodelo, escalable y bien indexada. Todo el proceso garantizó la portabilidad del modelo y su correcta auditoría técnica mediante la extracción del diccionario de datos físico con INFORMATION_SCHEMA

JMM

La construcción física de esta base de datos deja claro que pasar de un modelo teórico a un script SQL funcional requiere mucha rigurosidad técnica. MySQL demostró ser un motor muy completo para soportar la lógica de una plataforma interactiva, logrando un buen equilibrio entre la integridad de las relaciones tradicionales y la flexibilidad de las nuevas tecnologías de datos.

Finalmente, el proceso evidencia que el éxito de un despliegue no depende únicamente de tener un buen diseño entidad-relación, sino de conocer y adaptarse al motor de base de datos: automatizar la limpieza del esquema, cuidar el orden secuencial de los

comandos y dominar los tipos de datos físicos específicos del SGBD para sacarles el máximo provecho.

Conclusión Grupal:

El desarrollo colaborativo de este proyecto permitió evidenciar que la migración e implementación de una base de datos multimodelo no se limita a un cambio sintáctico entre dialectos SQL, sino a una comprensión profunda de la arquitectura interna de cada SGBD. La integración de estructuras relacionales tradicionales con atributos NoSQL (JSON/JSONB) y espaciales (IoT) demostró ser una solución viable para soportar la dinamización de datos en una red social estudiantil sin sacrificar la integridad ni el rendimiento.

GitHub:

https://github.com/johanmartinez927/RedSocial/tree/main/TAREA4

YouTube:

https://youtu.be/TnDpOs5NjlQ